GitHub Copilot e formação profissional: critérios para revisar sugestões de código

ISABELLA LUIZA DOS SANTOS SOUZA, ESPECIALISTA.[1]
JOSÉ KAILON TEIXEIRA DA SILVA[2]
CARLOS HENRIQUE NUNES FERREIRA BARBOSA[3]

RESUMO

O uso de um assistente de código traz uma questão prática para quem se prepara para atuar em desenvolvimento de sistemas: em que momento uma sugestão pode ser aceita como parte de uma solução? Este trabalho discute essa decisão a partir de estudos sobre o GitHub Copilot. O foco está no papel de quem lê, testa e incorpora o código, e não apenas na rapidez com que ele é produzido. A questão de pesquisa é como os benefícios e os limites descritos na literatura podem orientar a revisão das sugestões no trabalho profissional. O objetivo é discutir critérios para essa revisão, com base em dados sobre o uso e a segurança da ferramenta. A pesquisa é uma revisão narrativa, de natureza teórica e abordagem qualitativa, sem teste próprio do Copilot. Em 15 de outubro de 2026, foram feitas buscas na web por "GitHub Copilot", "usability" e "security", além de títulos e nomes de autores. Foram consultados textos completos em Harvard, UC San Diego e arXiv, com verificação dos DOI das publicações ACM e IEEE. O recorte admitiu estudos de 2021 a 2026. A seleção exigiu método explícito, acesso ao texto e relação com o tema; materiais de divulgação e cópias do mesmo estudo não entraram no conjunto. Três pesquisas de 2022 e 2023 foram escolhidas para comparar formas de uso, resultados e limites. Em um estudo com 24 pessoas, Vaithilingam, Zhang e Glassman (2022) observaram que a preferência pelo Copilot não se traduziu em diferenças estatisticamente significativas no tempo ou no sucesso das tarefas. Esse achado leva a separar a impressão de ajuda da prova de que uma tarefa foi bem resolvida. Já Barke, James e Polikarpova (2023) observaram 20 pessoas e descreveram usos de aceleração e exploração: em um caso, o usuário sabe como seguir; no outro, busca caminhos para resolver o problema. Assim, a mesma ferramenta pode ocupar papéis distintos, e o grau de familiaridade com a tarefa precisa entrar na análise da revisão. Esses estudos não fornecem uma regra única para medir o ganho de tempo em todo projeto. A segurança acrescenta outra exigência. Pearce et al. (2022) encontraram vulnerabilidades em cerca de 40% de 1.689 programas gerados em 89 cenários. A taxa se refere ao experimento, que inclui software e hardware, e não deve ser tomada

1 Graduação em Análise e Desenvolvimento de Sistemas. Especialização em Algoritmos e Lógica de Programação.
2 Centro Universitário Braz Cubas - Graduando em Análise e Desenvolvimento de Sistemas (2º semestre).
3 Centro Universitário Braz Cubas - Graduando em Análise e Desenvolvimento de Sistemas (2º semestre).

como risco geral de uso. O estudo avalia saídas de código, enquanto os outros analisam a interação de pessoas com o assistente. Apesar dessa diferença, o diálogo entre eles permite propor que a aceitação de uma sugestão passe por critérios claros: entender sua função, verificar sua relação com o requisito e buscar falhas que um teste de uso comum pode não revelar. Esses critérios são uma proposta desta revisão e não um resultado medido com alunos. Para a formação profissional, um exercício coerente seria pedir que o estudante explicasse por que manteve ou alterou uma sugestão e quais testes sustentam sua escolha. Isso daria visibilidade ao raciocínio usado na decisão. Essa avaliação deve considerar as exigências do projeto e os riscos de aceitar trechos que o usuário ainda não compreende. A análise tem limites: são apenas três estudos, com tarefas e amostras restritas, além de versões antigas do Copilot. Não é possível concluir que os mesmos resultados se repetem hoje ou em toda equipe. Também falta, nesse conjunto, acompanhar por mais tempo o custo das correções após a aceitação do código. Conclui-se que o assistente pode apoiar a busca de soluções, mas a decisão de usar sua saída precisa de critérios de qualidade e de uma revisão que possa ser explicada. Em estudos futuros, seria útil observar projetos completos e registrar tanto a ajuda recebida quanto o esforço gasto para corrigir e manter o que foi gerado.

Palavras-chave: GitHub Copilot. Formação profissional. Revisão de código.

REFERÊNCIAS

BARKE, Shraddha; JAMES, Michael B.; POLIKARPOVA, Nadia. Grounded Copilot: How Programmers Interact with Code-Generating Models. Proceedings of the ACM on Programming Languages, v. 7, n. OOPSLA1, art. 78, 27 p., 2023. DOI: https://doi.org/10.1145/3586030.

PEARCE, Hammond; AHMAD, Baleegh; TAN, Benjamin; DOLAN-GAVITT, Brendan; KARRI, Ramesh. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In: 2022 IEEE Symposium on Security and Privacy (SP). IEEE, 2022. p. 754-768. DOI: https://doi.org/10.1109/SP46214.2022.9833571.

VAITHILINGAM, Priyan; ZHANG, Tianyi; GLASSMAN, Elena. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In: CHI Conference on Human Factors in Computing Systems Extended Abstracts. New York: ACM, 2022. 7 p. DOI: https://doi.org/10.1145/3491101.3519665.